

Title of Issue

SQL Injection Vulnerability Allowing Authentication Bypass

Description

The application's login functionality is vulnerable to SQL Injection. User-supplied credentials are incorporated directly into a backend authentication query without parameterization, allowing an attacker to manipulate the query logic and bypass authentication entirely.

The application constructs an authentication query similar to the following:

```
SELECT * FROM users WHERE username = 'wiener' AND password = 'bluecheese'
```

By injecting SQL comment syntax into the username field, an attacker can comment out the password check and authenticate as any known user — including the `administrator` account — without supplying a valid password.

Affected Endpoint: `POST /login` **Vulnerable Parameter:** `username`

Steps to Exploit

1. Navigate to the application's login page at `/login`.
2. In the **username** field, enter the following payload:

```
administrator'--
```

3. In the **password** field, enter any arbitrary value (e.g., `anything`).
 4. Submit the login form.
 5. Observe that the application authenticates you as the `administrator` user without requiring a valid password.
-

Proof of Concept

Payload (username field):

```
administrator'--
```

Original Query:

```
SELECT * FROM users WHERE username = 'administrator' AND password = 'anything'
```

Injected Query:

```
SELECT * FROM users WHERE username = 'administrator'--' AND password = 'anything'
```

The single quote (`'`) closes the string literal for the username. The double-dash (`--`) comments out the remainder of the SQL query, including the `AND password = '...'` clause. The database therefore returns the `administrator` row without verifying the password, granting full administrative access.

- Entering `administrator'--` as the username with any password triggers a successful login.
- The session is established as the `administrator` user, confirmed by access to the admin panel.

 Login page with injection payload entered

 Successful authentication as administrator

 Lab completion confirmation

Impact

- Complete authentication bypass — any account can be accessed without valid credentials.
- Full administrative compromise of the application.
- Potential for privilege escalation, data exfiltration, and destructive actions.
- Breach of confidentiality, integrity, and accountability.
- Exposure of all user data accessible through the administrator account.

Mitigation / Remediation

1. **Use parameterized queries (prepared statements)** — never concatenate user-supplied input into SQL strings.
2. **Implement multi-factor authentication (MFA)** — reduces impact even if authentication logic is bypassed.
3. **Apply strict input validation** — reject usernames containing SQL special characters (`'` , `--` , `;`).
4. **Use an ORM framework** that handles query construction securely by default.
5. **Log and alert on unusual authentication patterns** — multiple failed attempts or unusual username formats.

CVSS Score

CVSS v3.1 Score: 9.8 (Critical) `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`

CVSS Justification

Metric	Value	Rationale
Attack Vector	Network	Exploitable remotely via the login form
Attack Complexity	Low	Single payload, no special conditions required
Privileges Required	None	No prior authentication needed
User Interaction	None	Attacker acts alone
Scope	Unchanged	Impact contained within the application
Confidentiality Impact	High	Full access to all data accessible by administrator

Integrity Impact	High	Administrator can modify any application data
Availability Impact	High	Administrator can delete data or disrupt service